ThoughtSpot — Security, Access Control, Admin at Scale, Auditing

Research for the Spiff design team. Sourced from docs.thoughtspot.com, developers.thoughtspot.com and thoughtspot.com/trust. Quoted text is ThoughtSpot's. Anything I could not confirm is marked UNVERIFIED. §10 Group B, §11's UI patterns and §12 are my reasoning, not ThoughtSpot features — flagged inline.

---

1. Users, groups, roles, privileges

docs.thoughtspot.com/cloud/latest/groups-privileges · /rbac · developers.thoughtspot.com/docs/rbac

Two coexisting models.

Legacy group-privilege model. Privileges attach to groups; users get the union — "If a user belongs to more than one group, they will have the highest level of the privileges from all the groups they belong to." Groups nest: "subgroup members automatically receive parent group privileges." One default group, All, "automatically includes every user," cannot be deleted, and members cannot be removed from it.

RBAC model. "A role is a collection of privileges," assigned to groups, not users — "Users inherit role privileges from the groups to which they are assigned." A group may hold several roles; effective privileges are the union. Two constraints: RBAC is off by default, requested via support, and once enabled cannot be disabled; and "Roles are unique to an Org and can be created only within the context of an Org."

Full RBAC privilege list (API enum / UI label):

API name	UI label
ORG_ADMINISTRATION	Can manage Orgs
USER_ADMINISTRATION	Can manage users
GROUP_ADMINISTRATION	Can manage groups
ROLE_ADMINISTRATION	Can manage roles
AUTHENTICATION_ADMINISTRATION	Can manage authentication
APPLICATION_ADMINISTRATION	Can manage application settings
SYSTEM_INFO_ADMINISTRATION	Can view system activities
BILLING_INFO_ADMINISTRATION	Can view billing information
CONTROL_TRUSTED_AUTH	Can enable or disable trusted authentication
TAGMANAGEMENT	Can manage tags
CAN_SETUP_VERSION_CONTROL	Can set up version control
CAN_MANAGE_ANALYST_STUDIO / CAN_ACCESS_ANALYST_STUDIO	Can manage / use Analyst Studio
A3ANALYSIS	Has SpotIQ privilege
DEVELOPER	Has developer privilege
JOBSCHEDULING	Can schedule for others
SYNCMANAGEMENT	Can manage sync settings
CAN_USE_SPOTTER / CAN_MANAGE_SPOTTER	Can use / manage Spotter
CAN_CREATE_CATALOG	Can manage catalog
SHAREWITHALL	Can share with all users
USERDATAUPLOADING	Can upload user data
BYPASSRLS	Can administer and bypass RLS
CAN_MANAGE_CUSTOM_CALENDAR	Can manage custom calendars
CAN_CREATE_OR_EDIT_CONNECTIONS	Can create/edit Connections
CAN_MANAGE_WORKSHEET_VIEWS_TABLES	Can manage data models
CAN_MANAGE_VARIABLES	Can manage variables

DATADOWNLOADING	Can download data
CAN_DOWNLOAD_VISUALS	Can download visuals
CAN_DOWNLOAD_DETAILED_DATA	Can download detailed data

Also documented: Can invoke Custom R Analysis, Can verify Liveboard, and legacy labels Can administer ThoughtSpot, Can administer Org, Can create SQL views, Can manage data. Built-ins: the Super Admin role ("all privileges previously held by administrators") and the ALL role, auto-assigned to the ALL group, granting the mandatory AUTHORING privilege.

Two things stand out. Download is split into three privileges — exfiltration is its own risk axis, separate from viewing. And SHAREWITHALL gates seeing other users' names: it lets a user "see the names of and share with users outside of the groups the user belongs to," so without it the directory itself is scoped.

2. Orgs / multi-tenancy

docs.thoughtspot.com/cloud/latest/orgs-overview

An Org "logically partitions a ThoughtSpot cloud instance into multiple tenant-specific environments." Isolated per Org: warehouse connections and data, groups (unique per Org, never shared), objects, and search indexing. Objects cannot be shared across Orgs.

Users are the exception — "users can belong to multiple Orgs, but at any given time can only access the data and objects of their current Org," and are "unaware of Orgs they don't belong to." Switching uses an Org switcher in the top navigation bar, next to the help icon, hidden entirely if you belong to one Org; URLs carry Org context parameters. A non-deletable Primary Org exists, whose admin is the cluster admin, working cross-Org from an "All orgs" section.

3. Row-level security

/cloud/latest/security-rls · /rls-rule-builder-reference · /security-rls-implement

Rule-based and expression-driven, defined on a table, inherited by Models. A rule is a boolean evaluated "for every row and group combination"; true means the group sees the row.

Variables: ts_groups (list of the user's groups — legacy string form ts_groups = 'east', list form 'single_group_name' in ts_groups), ts_username, and ts_groups_int (integer group IDs, "enables efficient filtering without string comparisons", e.g. group_id = ts_groups_int). These cannot be used as function arguments — substr(ts_groups,0,3) is invalid. Outer joins need explicit null tolerance: ts_groups = customer_id_fact or is_null(customer_id_fact).

Functions span conversion (to_bool, to_date), date (add_days, start_of_month, quarter_number, diff_days, is_weekend), text (concat, contains, substr, sounds_like, spells_like, similarity), set membership (state in { 'texas', 'california' }) and if…then…else.

UI: Data workspace → double-click table → Row security → + Add row security → Rule Builder. Name it, type an expression, open the Rule Assistant panel for operators/functions/examples, get a green indicator and "Good to go" when valid, pick a join path if several exist, Save.

Testing — a real gap. No "preview as user" is documented in the Rule Builder. The docs advise you to "sign in as users in different groups" and search for data the test user can and can't access.

Two related settings: Strict RLS is "the default setting applied to clusters… ensures that if an RLS rule is defined then this is always included in any query," disable-able only for performance at scale. And search suggestions respect RLS only "when the rule is defined explicitly on the table that the column is derived from" — with passthrough security they recommend disabling indexing on sensitive columns. Autocomplete is a leak surface.

4. Column-level security

/cloud/latest/security-data-object · /share-source-tables

Two generations. Share-based: from the Data tab choose Entire Table or Specific Columns; the latter activates CLS, and each column is shared separately per user/group. Recommended now: column security rules at the table level, which "will automatically be inherited to associated Models" and don't require Strict CLS. Access is a per-group toggle, "Has access" / "No access" — and "You cannot specify column security rules at user level." No column masking is documented (UNVERIFIED whether it exists).

Two global modes. Permissive (default): "when someone shares an object with you, you can see all the data it uses, regardless of explicit permissions to the parent object data." Advanced Security: "Unless the user has explicit permissions to the entire stack of parent objects, they cannot see the data in the child object." Note the default widens data access on share.

5. Object-level sharing

Sources as above, plus /liveboard-granular-permission

Objects are "tables, columns in tables, Models, Liveboards, and saved Answers." Levels are Can View and Can Edit (edit on a table = rename/alter/delete plus column management). The share dialog carries "Give view access to underlying data sources." Sharing is transitive by default: "A user can automatically share objects with anyone else in the groups to which they belong," and recipients "can further distribute access to others." Locking that down needs a separate, now-deprecated privilege "Cannot copy or edit existing Liveboards," which strips Share, Pin, Download, Edit, Copy and edit, and Copy embed link. The docs name the resulting risk: restricted columns can be "inadvertently shared outside intended groups."

6. Access requests

/cloud/latest/share-request-access

Three request types, each reachable from a dead end rather than a helpdesk ticket: request view access (a Request Access page with optional message and Request access button), request edit access (a locked icon beside the object name), and request data access (a locked icon whose dropdown lists the required data sources).

Notifications go wide: the owner, anyone with edit access, all cluster administrators, and ThoughtSpot support. "Any of the users who got the request then shares the Liveboard or Answer with the user." The requester is told who owns the object. What this is not: there is no approval queue, SLA, expiry, or audit trail of grants-via-request. It is email plus a manual share.

7. Audit logging and admin analytics

/cloud/latest/audit-logs · /system-liveboards · /user-adoption.html

Every security event carries Event ID, human-readable description, UTC timestamp (yyyy/mm/dd:hh:mm:ss), user ID, public IP address, plus event-specific fields. Retention: "Security events remain within the system for 30 days." Access is push to SIEM "in near real-time" at 5-second intervals (HTTP, Splunk, Azure Log Analytics, Datadog) or pull via the Audit Logs API (admin privileges required).

Documented event names, verbatim:

- Auth: LOGIN_SUCCESSFUL, LOGIN_FAILED, LOGOUT_SUCCESSFUL, LOGOUT_FAILED, ACCOUNT_LOCKED, UPDATE_PASSWORD, UPDATE_PASSWORD_FAILED,

AUTH_TOKEN_CREATED_SUCCESSFULLY, FAILED_TO_CREATE_AUTH_TOKEN
- Create: CREATE_ANSWER, CREATE_PINBOARD, CREATE_MODEL, CREATE_WORKSHEET,
CREATE_TABLE(S), CREATE_VIEW, CREATE_SQL_VIEW, CREATE_JOIN, CREATE_RELATIONSHIP,
CREATE_CONNECTION, CREATE_CONNECTION_ATTEMPTED, CREATE_IMPORTED_TABLE,
CREATE_RLS_RULE
- Update: UPDATE_ANSWERS, UPDATE_PINBOARDS, UPDATE_MODEL, UPDATE_VIEW, UPDATE_JOIN, UPDATE_RELATIONSHIP, UPDATE_RLS_RULE, EDIT_TABLE, EDIT_WORKSHEET,
EDIT_SQL_VIEW, EDIT_CONNECTION, EDIT_CONNECTION_ATTEMPTED, EDIT_IMPORTED_TABLE
- Delete: DELETE_ANSWERS, DELETE_PINBOARDS, DELETE_MODEL, DELETE_VIEW, DELETE_TABLE, DELETE_WORKSHEET, DELETE_SQL_VIEW, DELETE_JOIN, DELETE_RELATIONSHIP, DELETE_CONNECTION, DELETE_IMPORTED_TABLE, DELETE_RLS_RULES
- Identity/entitlement: USERS_CREATED, USERS_MODIFIED, USERS_DELETED, USER_ACTIVATE, USER_INVITED, USER_GROUPS_CREATED, USER_GROUP_MODIFIED, USER_GROUPS_DELETED, PRINCIPALS_IN_GROUP_UPDATE, PRIVILEGE_CHANGES, ROLE_CREATED,
ROLE_UPDATED, ROLE_DELETED, ROLES_ASSIGNED, ROLES_REMOVED, ROLES_IMPORTED
- Sharing: SHARE_OBJECTS — the only sharing event, notably coarse
- Data movement: CSV_UPLOAD_STARTED, CSV_UPLOAD_FINISHED, DATA_UPLOAD_CONFIGURED
- Org: ORG_CREATION_SUCCESSFUL/FAILED, ORG_DELETION_SUCCESSFUL/FAILED, ORG_SWITCH_SUCCESSFUL/FAILED, ORG_ACCESS_GRANTED_TO_USER
- Lifecycle/billing: TRIAL_USER_CREATE/DELETE/END/EXPIRE/EXTEND, TEAM_CHANGE_SUBSCRIPTION, USER_CHANGE_SUBSCRIPTION, TEAM_EDITION_USER_DELETE/EXPIRE

Conspicuously absent: no view, query or download event. Read and export activity lives in a separate system — the monitoring Liveboards — not the audit stream. That split-brain is the single biggest weakness here.

System Liveboards (default-visible only to Can administer ThoughtSpot, refreshed hourly): User Adoption, Object Usage, Performance Tracking, Connections, Credit Usage, TS Stats: Latency Visualizations, How Users Are Searching Answers, AI and BI System Liveboard, Spotter Conversations.

The User Adoption Liveboard is the reference design: DAU Last 4 Weeks, WAU Last 4 Weeks, MAU Last 30 days, Weekly/Daily/Hourly Active Users, Count of Current Liveboards, Count of Current Answers, ThoughtSpot Champions Last 30 Days, Top 10 Liveboard Consumers Last 30 days, Top 10 Adhoc Searchers, Count of visualizations generated, last month, Count of object interactions, last month, Popular Liveboards Last 30 Days, Top 10 Liveboards Ratio by Views, Top 10 Liveboards Ratio by Users, User Actions Last 4 Weeks, Abandoned Answers, Abandoned Liveboards, Inactive Users (KPI + table), User Profile, and Hours Saved by Monthly Query Start Date and Task split by Drill down / Explore / Search Data / SpotIQ auto analyze. Filterable by user. Champions, Abandoned and Inactive Users are the three worth stealing — they imply an action.

8. Impersonation / "view as"

Partially verified. There is no documented in-product admin "view as user" toggle and no RLS preview-as. A developer path exists: trusted auth "can also be used for back-end REST API processes that need to impersonate an individual user to retrieve a filtered data

response," and with the secret key such processes "can request a token for any user, and then use the returned token as a login token using session/login to create a long-lived session as that user" (developers.thoughtspot.com/docs/trusted-auth). Gated by CONTROL_TRUSTED_AUTH
(Super Admin only). So: impersonation is an API primitive with no admin UI on top. UNVERIFIED whether impersonated sessions are distinguishable in the audit log.

9. AI governance

/cloud/latest/spotter-security · /spotter-data-handling · thoughtspot.com/trust/enterprise-grade-ai

The load-bearing claim: "If a user cannot see a specific row or column in ThoughtSpot due to row-level or column-level security, that data is never sent to the Large Language Model." Alongside "only the specific data required to answer your question is accessed—never entire tables or databases," and a symmetry rule — "if a column is hidden to restrict it from being shared with the LLM, it is also hidden from standard ThoughtSpot Search."

What does go to the LLM: "Model column names and descriptions," "Sample data values (to ensure prompt accuracy)," contextual metadata, and for Spotter 3 "data from a data warehouse query response"; multi-step analytics "creates additional LLM prompts, sending CDW query results to LLMs."

Default-off: "All AI-powered features – including Spotter, SpotIQ, AI Highlights, and AI Assist – are disabled by default," enabled "for specific models or columns." Enablement is per-model and per-column, not a global switch.

Retention: Azure OpenAI — "Customer data is not persisted or cached," but Spotter 3 "query results are cached for up to six hours by default"; with saved chats on, results persist for the chat history retention period, maximum 180 days. Vertex AI — not persisted. Neither trains on customer data. Spotter Conversations events retain 180 days. TLS in transit; providers pass "ThoughtSpot's Vendor Security Risk Assessment process."

Admin paths: chat history at Admin > ThoughtSpot AI > Spotter 3 capabilities; version at Admin > ThoughtSpot AI > Spotter version; column exclusion via the data model.

Transparency: search tokens. "Since search tokens are always visible to you, they provide transparency into what information is being fed into the AI" and let users "validate Spotter's results" — a legible intermediate representation between question and query.

Feedback moderation lives in the Spotter Conversations Liveboard: Active users (weekly), Conversations (weekly), Questions asked (weekly), Conversations with feedback (weekly), Most active users, Users providing feedback, Feedback Response, Conversations by origin / by Model / by Liveboard, Avg Conversation Length, Conversation length distribution, plus Total conversations, Downvoted conversations, Conversations with downvoted responses and a Complete conversations log. A User Feedback Rating column records votes. Admins "identify problematic areas through downvoted responses" and "refine datasets and adjust Spotter coaching."

Trust badging — Verify Liveboard (/cloud/latest/liveboard-verify): a Can verify privilege; users request verification from the More options menu; verifiers get a notification, click View Liveboard, then Approve. Verified objects show a badge, a sortable Verified column, and verifier name + date under "Show Liveboard details." Editing a verified Liveboard raises a warning banner that content changed and may need re-verification.

---

10. An exhaustive audit-event vocabulary

Group A — confirmed ThoughtSpot events (§7 above): auth; full CRUD across answers, dashboards, models, tables, views, SQL views, joins, relationships, connections; identity and entitlement changes; security-rule CRUD; SHARE_OBJECTS; data ingest; tenancy; subscription

lifecycle. Note their attempt-vs-success pattern on risky operations (CREATE_CONNECTION_ATTEMPTED beside CREATE_CONNECTION) — worth copying wholesale.

Group B — my reasoning; gaps Spiff should close. Not ThoughtSpot features.

- Read and egress (their biggest gap): REPORT_VIEWED, QUERY_EXECUTED (SQL, row count, duration, warehouse cost), RESULT_SET_EMPTY_DUE_TO_RLS, DOWNLOAD_INITIATED / DOWNLOAD_COMPLETED (format, rows, columns, bytes), PRINT_OR_SCREENSHOT_EXPORT, EMBED_LINK_CREATED, PUBLIC_LINK_CREATED / REVOKED, BULK_EXPORT_THRESHOLD_EXCEEDED, API_DATA_PULL.
- Denials, not only grants: ACCESS_DENIED_OBJECT, ACCESS_DENIED_COLUMN, ACCESS_DENIED_ROW_FILTER_APPLIED, PRIVILEGE_CHECK_FAILED. A log of successes alone cannot answer "who is probing what."
- Access-request lifecycle: ACCESS_REQUESTED, ACCESS_REQUEST_APPROVED / DENIED / EXPIRED, ACCESS_GRANT_EXPIRED, ACCESS_REVIEW_STARTED / COMPLETED, ENTITLEMENT_RECERTIFIED, ENTITLEMENT_REVOKED_AT_REVIEW.
- Delegation: IMPERSONATION_STARTED / ENDED (actor and subject as separate fields on every downstream event), RLS_BYPASS_USED, BREAK_GLASS_ACCESS_GRANTED, SERVICE_ACCOUNT_ACTED_ON_BEHALF_OF.
- Automation / run-as — core to Spiff: AUTOMATION_CREATED/EDITED/PAUSED/DELETED, AUTOMATION_CLONED_ON_SHARE, AUTOMATION_RUN_STARTED/SUCCEEDED/FAILED, AUTOMATION_RUN_SKIPPED_OWNER_DEPROVISIONED, AUTOMATION_PERMISSION_RECHECK_PASSED/FAILED, AUTOMATION_OWNER_TRANSFERRED, SCHEDULED_DELIVERY_SENT (per-recipient scope), DELIVERY_SUPPRESSED_EMPTY_FOR_VIEWER.
- AI: AI_PROMPT_SUBMITTED, AI_TOKENS_RESOLVED, AI_QUERY_GENERATED, AI_ANSWER_RETURNED, AI_ANSWER_FEEDBACK_UP/DOWN, AI_ANSWER_CORRECTED_BY_USER, AI_REFUSED_OUT_OF_SCOPE, AI_HALLUCINATION_FLAGGED, AI_FEATURE_ENABLED_FOR_MODEL/COLUMN, AI_COLUMN_EXCLUDED_FROM_LLM, LLM_PROVIDER_CALL_MADE (provider, model version, payload class, latency), CHAT_HISTORY_RETENTION_CHANGED.
- Trust & content lifecycle: CONTENT_VERIFIED / VERIFICATION_REVOKED, VERIFIED_CONTENT_EDITED_AFTER_VERIFICATION, DEPRECATION_FLAGGED, OWNERSHIP_TRANSFERRED, CONTENT_ARCHIVED_FOR_DISUSE, DEFINITION_CHANGED_ON_CERTIFIED_METRIC.
- Config posture: SECURITY_SETTING_CHANGED (before/after), STRICT_RLS_DISABLED, ADVANCED_SECURITY_MODE_TOGGLED, SSO_CONFIG_CHANGED, SCIM_SYNC_RUN, RETENTION_POLICY_CHANGED, AUDIT_EXPORT_PERFORMED, AUDIT_LOG_ACCESSED.

Field schema for every event: event id · type · UTC timestamp · actor id · effective subject id (differs under impersonation/run-as) · org · source IP · client (web/API/scheduler/embed) · target id + type + name · before/after on mutations · outcome (success/failure/denied) · reason code · correlation id · session id. ThoughtSpot captures about the first six.

11. Managing hundreds of users

What ThoughtSpot ships (/software/latest/admin-portal-users): Admin Console → Users. + add user to create. Hover a username to reveal a checkbox, multi-select, then Add users to groups or Delete. Cross-Org bulk add lives in the cluster admin's All orgs section. SCIM provisioning is supported alongside SAML group mapping and OIDC.

UNVERIFIED: column headers, pagination and filtering are not documented — the admin list looks thin.

Patterns worth building (mine, except where noted):

- Never assign to individuals. Their structural choice — roles bind to groups, RLS and CLS are group-scoped, "you cannot specify column security rules at user level" — is the most scalable decision in the product. Make the user row read-only for entitlements.
- Hover-reveal checkbox with a persistent selection bar that survives filtering and paging: "142 users selected" plus Clear.
- Faceted filter rail, not a lone search box: group, role, org, last-active bucket, provisioning source (SSO/SCIM/manual), status, licence, has-RLS-exemption.
- Saved segments — "Inactive 90+ days", "Admins", "Externally provisioned".
- Diff preview before bulk apply: "adds 84 users to GST Analysts, grants 3 new privileges, 12 already members, 2 conflict" — then an undo window.
- Effective-permissions inspector on a user: not "member of 6 groups" but the resolved answer — every privilege, the group that granted it, applicable RLS rules, hidden columns. Union-of-groups semantics make this impossible to compute by eye.
- Reverse view on the object: "Who can see this?" — groups expanded to a headcount and named list, warning when a share crosses an audience threshold.
- Entitlement review campaign: per-owner worklists, each row Keep / Revoke / Delegate, justification required on Keep, progress tracking, auto-revoke deadline.
- Orphan and drift sweeps: objects owned by deactivated users, empty groups, roles with no groups, RLS rules referencing dead groups, automations whose owner lost data access.
- Bulk ownership transfer as a first-class step in offboarding.
- Dry-run impersonation ("view as this user / as a member of this group") as the standard verification after any bulk change — the thing ThoughtSpot lacks.

12. Design lessons for Spiff

1. Group-only entitlements. Copy the constraint literally: roles, RLS and CLS bind to groups, never individuals. It is the only model that survives a thousand users.
2. Make "view as" a first-class surface. Their advice for testing RLS is "sign in as users in different groups" — a gap, not a pattern. A Preview as [user | group] control on every report, rule and automation, with a loud banner and an IMPERSONATION_STARTED event, is cheap to mock and instantly reads as governance.
3. Log reads and exports, not just writes. Their security log has full CRUD but no view, query or download event. Spiff needs one stream answering "who saw what, and what left the building."
4. Model denials and attempts, not only successes — extend their CREATE_CONNECTION_ATTEMPTED instinct to ACCESS_DENIED_.
5. Sharing must not widen access — and must say so on screen. Their default is permissive: sharing an object shares its data "regardless of explicit permissions to the parent object." Spiff inverts this, so show it: "Ana will run this herself. She'll see only her own permitted rows — you may see more."
6. Show per-viewer scope inside the share dialog. As you add a recipient, render what they will see: row count, hidden columns, empty-result warning. Visible proof of "re-runs per viewer, scoped to that person."
7. Clone-on-share needs its own event and visual language: AUTOMATION_CLONED_ON_SHARE plus a lineage chip ("copied from Oren's GST weekly, 3 Mar") on every clone.
8. Split download into separate permissions. CAN_DOWNLOAD_VISUALS vs CAN_DOWNLOAD_DETAILED_DATA is a genuinely useful line — a picture is not a dataset.
9. Scope the user directory itself. SHAREWITHALL gates whether you can even see names outside your groups. In a GST division the org chart is information.
10. AI default-off, per-model and per-column. That granularity is the governance story, and it is a screen: a model page with a per-column Visible to AI toggle.

11. Steal the search-token transparency trick. Show the interpretation — filters, metrics, time grain — as editable chips between question and query; log AI_TOKENS_RESOLVED and AI_ANSWER_CORRECTED_BY_USER.
12. Feedback is a governance instrument, not a smiley face. Downvotes should roll into an admin view of Downvoted conversations routed to the model owner as a work queue.
13. Verification with decay. Copy the Can verify privilege, the badge, the sortable Verified column — especially the warning banner when a verified object is edited.
14. Autocomplete is an access-control surface. They warn suggestions leak values from restricted columns; scope every suggestion to the viewer.
15. Turn the dead end into a request. The locked-icon → Request access → dropdown of required data sources pattern is excellent. Add a real approval queue, decision record and expiry — theirs is email plus a manual share.
16. 30 days is not enough. Security-event retention is 30 days (Spotter conversations 180). For a GST context, state a longer retention and surface it in the UI.

13. Screens/components worth stealing

- Rule Builder with inline Rule Assistant, live validation and a green "Good to go" state — formula editing with training wheels.
- Row security tab on the table object — security lives on the data, not in an admin silo, and inherits to Models automatically.
- Per-group column matrix with Has access / No access toggles: rows = columns, columns = groups. One screen, whole policy.
- Share dialog with Can View / Can Edit plus "Give view access to underlying data sources" — it makes the data-vs-object distinction visible at the decision point.
- Locked-icon affordance on inaccessible content, opening a request with a dropdown of the specific missing data sources.
- Org switcher in the top nav beside the help icon, hidden for single-org users — tenancy that costs nothing when unused.
- User Adoption Liveboard, specifically ThoughtSpot Champions, Inactive Users, Abandoned Answers / Liveboards, and Hours Saved by task.
- Spotter Conversations Liveboard as the AI-governance console: Conversations with feedback, Downvoted conversations, Conversations by Model, Complete conversations log.
- Verified badge + sortable Verified column + post-edit re-verification banner.
- Admin > ThoughtSpot AI page (Spotter version, Spotter 3 capabilities) — one screen answering "what is the AI allowed to do here."
- Hover-reveal multi-select on the Users list with contextual Add users to groups / Delete.

Verification notes

UNVERIFIED: column masking (vs hiding) in CLS; exact Users-list columns, pagination and filtering; whether impersonated sessions are distinguishable in audit logs; SCIM IdP list and synced attributes; Audit Logs API rate limits. Not ThoughtSpot features: all of §10 Group B, most of §11's patterns, and §12 — my reasoning for Spiff. ThoughtSpot docs are versioned (/cloud/latest/, /cloud/26.8.0.cl/, /software/latest/); the granular-Liveboard privilege is explicitly deprecated in the new Answer experience.